Certified Chopping for Agent Tool Workflows: Admitting Plan Revisions over Committed Effects

Preliminary research draft | 10 September 2026 | Author list to be confirmed

STATUS: the mechanism, theorems, verification and runtime described here are implemented and reproducible from the accompanying artifact. Reasoning time in the throughput benchmarks is a calibrated delay rather than model inference; the real-agent study is a small enforcement experiment, not a performance benchmark. Related-work coverage is a targeted primary-source search, not an exhaustive systematic review.

Abstract

Concurrent AI agents read and write shared application state through tools, and they interleave badly. Wrapping each tool call in a transaction does not make an agent's intent atomic, while holding a transaction across an entire reason-and-act workflow serializes long intervals of private computation. Transaction chopping is the classical answer to exactly this tension, but it assumes the transaction programs are fixed before execution. Agents are not fixed: they revise their plans after observing tool results, by which time earlier pieces have already committed and their effects are visible.

We study the resulting problem, which we call prefix-aware recertification: deciding whether a mid-flight plan revision can be admitted when the past can no longer be rechopped. Our central result is a complete characterization. Because coarsening never introduces an SC cycle and sealing each workflow's uncommitted remainder into a single piece is the coarsest repair available after commitment, a revision is admissible under future-only repair if and only if the sealed plan is SC-cycle-free. The corollary is a genuine impossibility result: when the residual cycle passes through a piece that has already committed, no future-only repair exists, and quiescing every other workflow does not help. We also show admission decisions must be serialized, since two revisions that are individually admissible can be jointly unsafe.

We implement the mechanism in a runtime that mediates every tool access, enforces certified footprints fail-closed, executes pieces under deadlock-free strict two-phase locking, models predicate reads as ranges so phantoms are visible to certification, and commits each piece together with its durable progress record. Bounded exhaustive verification against a brute-force serializability oracle covers 4,352 static plans, 3,000 revision scenarios and 2,000 range scenarios with zero unsound admissions and zero violations of the characterization. On six workloads against four equivalent-guarantee baselines, certified chopping is up to 7.4x faster than a whole-workflow database transaction and 2.4x faster than whole-workflow two-phase locking where the structure permits chopping, and correctly collapses to whole-workflow locking where it does not, while unchecked chopping violates serializability and application invariants in every seed of three workloads. Two negative results are central: only 4 to 37 percent of mid-flight escalations into shared state are admissible, and certification cost can exceed the execution savings it enables. A small study with real language-model agents shows four concurrent agents losing three of four updates without certified boundaries, and none with them.

1. Introduction

Consider four agents that each reserve one unit of a shared resource. Each reads the counter, reasons briefly about whether to proceed, and writes back the incremented value. We ran exactly this with four concurrent language-model agents against a shared database through a mediated tool interface. All four read zero, all four wrote one, and the final counter was one instead of four: three lost updates, with no agent behaving incorrectly in isolation. The anomaly is not a reasoning failure; it is a concurrency-control failure, and no amount of prompt engineering removes it.

The obvious fix is to make the whole workflow a transaction. That works and is a strong baseline, but it holds conflicting resources across the agent's reasoning, which is the expensive part. In our measurements a whole-workflow database transaction is the slowest safe policy on every workload we tried, between 3.1x and 7.6x slower than the best safe alternative. The opposite fix, treating each tool call as its own transaction, is fast and wrong.

Transaction chopping resolves this tension for classical workloads by splitting a transaction into pieces that may execute and commit independently, provided a static graph over the pieces contains no cycle mixing sibling and conflict edges [1]. The theory is mature, and recent agent systems have independently converged on transaction-shaped abstractions for tool use, scheduling and recovery [7, 8, 9, 11, 12]. Applying chopping to agents is therefore not, by itself, a contribution.

What actually breaks when agents replan

The classical theorem assumes the set of transaction programs is known before execution. An agent violates this assumption in a specific, consequential way: it decides what to do next after seeing what it just read. A support agent reads an order, and only then discovers that the resolution requires touching a shared gift-card balance. By that point the read has committed and its result has been returned. The plan the certifier approved is no longer the plan being executed.

One can recertify the revised plan, and that is sound. The difficulty is what to do when recertification fails. Before execution, a failed certification is always repairable: merge pieces until, in the limit, each workflow is a single piece with no sibling edges. After a prefix has committed, that repair is unavailable, because merging a committed piece with a future one is not a physical operation. The question is whether some weaker repair always exists. We show it does not, and we characterize exactly when it does.

Contributions

1. We formalize prefix-aware recertification: admitting a plan revision against a batch in which some pieces have already committed, and where only the uncommitted suffix may be rechopped.

2. We prove a complete characterization (Theorem 2): a revision is admissible under future-only repair if and only if sealing every workflow's uncommitted remainder into one piece yields an SC-cycle-free plan. This gives a cheap, exact admissibility test and an algorithm that returns the finest certified chopping it can find.

3. We prove the corresponding impossibility result (Corollary 1) and show that waiting for concurrent workflows to finish does not rescue an inadmissible revision, because committed pieces retain both their sibling and their conflict edges. We also show admission must be serialized (Theorem 3).

4. We implement a runtime that makes the certificate's assumptions true rather than assumed: mediated access, fail-closed footprint enforcement, range footprints for predicate reads, deadlock-free piece-level strict two-phase locking, and atomic commit of each piece with its progress record.

5. We verify the mechanism exhaustively against brute-force ground truth, and evaluate it on six workloads against four equivalent-guarantee baselines with real threads and real wall-clock time, including crash injection, recovery, and a real-agent enforcement study.

We are explicit about what we do not claim. We do not claim a new isolation theorem for static choppings, a general ACID protocol for agents, or that this is the first transactional treatment of agent tool use. Our contribution is the revision-admission problem and its solution, plus an implementation and evaluation honest enough to show where the approach does not pay.

2. Model

An intent is the unit an application wants to be atomic and isolated: cancel this order and refund the customer. An intent is not an agent. Several agents can cooperate on one intent, and one long-lived agent can execute many intents, so the transaction identifier follows the delegated work rather than the process. In this paper each intent is one ordered workflow; distributed, internally parallel intents are future work.

A batch B is a finite set of workflow invocations. Workflow w has a plan, an ordered list of pieces, and each piece groups consecutive steps. A step performs mediated tool operations. Pieces of one workflow execute in order, and each piece commits before the next begins. Distinct invocations of the same program are distinct vertices; a single representative instance would miss self-interference between two concurrent invocations of the same program.

Each piece carries a declared footprint: a set of accesses, each an exact key or a half-open key range, marked read or write. A range models a predicate query, and a write to a key inside a declared range conflicts with a read of that

range, which is what makes phantom insertions visible to certification. Two accesses conflict when their items overlap and at least one is a write.

$$\text{conflict}(p, q) \iff \text{some } a \text{ in } F(p), b \text{ in } F(q) \text{ overlap and at least one is a write}$$

The SC graph over a plan is undirected. Its vertices are pieces. An S edge joins every pair of pieces of the same invocation, so each workflow's pieces form a clique. A C edge joins conflicting pieces of different invocations. The forbidden object is a simple cycle containing at least one S edge and at least one C edge. Pure S cycles and pure C cycles are permitted; a mixed closed walk is not sufficient. We detect mixed simple cycles by computing vertex-biconnected edge blocks and rejecting any block containing both labels, which is equivalent because any two edges of a biconnected block lie on a common simple cycle.

The runtime obligations are substantive and we implement all of them. Each piece must itself execute serializably; several atomic tool calls under one label are not an atomic piece. Program order between pieces must be respected. Declared footprints must bound actual accesses, enforced rather than trusted. Pieces aborted by contention must be retried to completion. These are the classical execution rules [1] restated for a tool runtime.

What a certificate does and does not buy

A certificate is a statement about isolation under a declared model. It is not atomicity: independently committed pieces leave a visible prefix if the process dies, which we demonstrate and then repair with durable continuation. It is not consistency: serializability preserves invariants only if each workflow is individually correct. It is not real-time ordering: conflict serializability does not imply strict serializability. It says nothing about observations outside the modeled state, including messages already sent. Irreversible external effects remain irreversible; a graph cannot create rollback for a service that does not offer it.

3. Prefix-aware recertification

Let plan P be certified and executing. The frontier phi records, for each workflow w, how many of its pieces have committed. Workflow r proposes to replace its uncommitted suffix with a new sequence, giving a revised plan P'. The runtime must decide whether to admit P', and if the proposed chopping is too fine, whether it can be repaired.

3.1 Soundness of recertification

Theorem 1 (Inherited isolation). *If P' is SC-cycle-free, its committed prefix is identical to the prefix actually executed, and the runtime satisfies the execution rules, then the resulting workflow history is conflict-serializable.*

Proof. The pieces that already ran are, by construction, exactly the prefix pieces of P', with the footprints they declared, and enforcement guarantees their actual accesses were inside those footprints. The observed execution is therefore a legal history of the chopping P' under the execution rules. The classical chopping theorem [1] applies to P' and yields serial equivalence with respect to the workflows of P'. This is an application of established theory to a revised plan, not a new isolation theorem.

The force of Theorem 1 is that recertification must cover the whole batch including committed pieces. Certifying only the new suffix is unsound, because it discards exactly the conflicts that close a cycle through the past.

3.2 What repairs remain after commitment

Before execution, a rejected chopping is repaired by merging pieces. After commitment, only the uncommitted suffix may be regrouped. Call P'' a future-only coarsening of P' at phi when the first phi(w) pieces of every workflow are unchanged and the remaining pieces are grouped into contiguous runs. Define seal(P', phi) as the future-only coarsening that merges each workflow's entire remainder into one piece.

Lemma 1 (Sealing is coarsest). *Every future-only coarsening of P' at phi is a refinement of seal(P', phi).*

Proof. Both agree on the committed prefix. Sealing places each workflow's whole remainder in a single piece, and any other future-only coarsening partitions that same remainder into contiguous runs, which is precisely a refinement of the single piece.

Lemma 2 (Cycles persist under refinement). *If a chopping contains a mixed simple cycle, so does every refinement of it.*

This is the classical non-removability property of chopping [1]: splitting pieces can only add S edges and redistribute C edges among the parts, so an existing mixed cycle survives. We check it empirically as well, over 1,023 refinements drawn from 258 scenarios whose sealing is uncertifiable, with no counterexample.

> **Theorem 2 (Admissibility characterization).** *A revision is admissible under future-only coarsening if and only if seal(P', phi) is SC-cycle-free.*

Proof. If sealing is cycle-free, sealing is itself a certified future-only coarsening, so the revision is admissible. Conversely, suppose sealing contains a mixed simple cycle. By Lemma 1 every future-only coarsening is a refinement of sealing, and by Lemma 2 every such refinement contains a mixed simple cycle. Hence no future-only coarsening certifies.

Theorem 2 turns admissibility into a single cheap test on the coarsest candidate, and it tells the runtime that if that test fails, searching the exponentially many intermediate choppings is pointless. When the test passes, we still want the finest certified plan rather than the coarsest, so the algorithm certifies the proposal directly, and only if that fails runs a guided merge restricted to indices at or beyond the frontier.

> **Corollary 1 (Impossibility of future-only repair).** *If every mixed simple cycle in seal(P', phi) passes through an S edge incident to a committed piece, then no future-only repair exists, and the runtime must refuse the revision or compensate the committed prefix.*

The practically important part of Corollary 1 is that waiting does not help. One might hope that letting all other workflows finish would dissolve the conflict. It does not: a workflow that has fully committed still contributes its S clique and its C edges, because those interleavings really happened. Concretely, suppose W0's committed piece read x and W1 committed a piece that wrote x and read y. If W0 now proposes to write y, the cycle W0-prefix, W1, W0-suffix is mixed and passes through W0's committed read. Our verifier confirms this instance is rejected, that the single available future-only chopping does not certify, and that the plan genuinely admits a non-serializable execution. The runtime's options are to refuse the revision, or to run a compensating action, which is an application-level obligation rather than a graph property.

3.3 Admission must be serialized

> **Theorem 3 (Serialized admission).** *Two revisions that are each admissible against a common snapshot may be jointly inadmissible, so admission decisions must be evaluated and applied against a single serialized plan snapshot.*

Proof by counterexample, machine-checked in the artifact. W0 has committed a read of x and W1 a read of y. W0 proposes to write y and W1 proposes to write x. Each revision alone is admitted. Applied together the plan is rejected, and it does in fact admit a non-serializable execution. Our runtime therefore evaluates and installs revisions under a single admission lock, which also fixes the frontier used for sealing.

3.4 Algorithm

Admission proceeds in three steps. Certify the proposed revised plan; if it passes, install it unchanged. Otherwise test seal(P', phi); if that fails, refuse, returning the witness cycle and the workflows it implicates so the application can choose compensation or an alternative plan. Otherwise run guided coarsening restricted to the uncommitted suffix, repairing every mixed block in each pass and choosing, per workflow, the interval whose merge adds the least footprint-duration area. Each pass strictly reduces the piece count, so the procedure terminates, and by Theorem 2 it must succeed once sealing has certified.

4. Runtime

The runtime is about 1,470 lines of Python over SQLite in write-ahead-logging mode. Its job is to make the certificate's hypotheses true.

Mediation and fail-closed enforcement. Every read, scan, write and delete passes through an adapter that checks the access against the manifest of the currently executing piece. An access outside the manifest is refused before it reaches the database, buffered writes are revalidated against the manifest at commit, and the piece is aborted. Enforcement does not depend on the agent's cooperation, which matters because a model-generated description of a

tool is not evidence about that tool's effects. The trust boundary is explicit: workflow bodies that run in-process are trusted not to bypass the adapter and reach the database directly, whereas the agent driving them is not. The real-agent study in Section 6.7 exercises the untrusted case, where the caller is a separate process that can only reach state through the mediated tool.

Piece isolation. A piece acquires locks for its entire declared footprint atomically, in one wait, before executing. Because nothing is acquired incrementally while holding locks, the lock manager is deadlock-free by construction, for chopped pieces and for whole-workflow conservative two-phase locking alike. Read and write modes are distinguished, and ranges lock intervals so that an insert into a scanned range is blocked.

Durable continuation. Each piece's buffered writes and the workflow's progress record, including its local variables, are written in a single database transaction. A crash inside that transaction rolls back both, so pieces are atomic. A crash after it leaves a durable marker from which recovery replays the remaining pieces forward. This is the recovery obligation that chopping requires and that a certificate alone does not provide.

Baselines. To keep the comparison about mechanism rather than guarantees, all policies share the same store, workflow code, footprints and reasoning delays. Whole DB transaction runs the entire workflow inside one exclusive database transaction. Whole-workflow 2PL holds the union footprint for the workflow's lifetime and commits once. Whole-workflow OCC runs unlocked, then validates every read version and re-executes every declared range scan before committing, so it is not vulnerable to phantoms, and retries the whole workflow on conflict. Unchecked chopping executes the proposed fine pieces with no certification and is included as an unsafe control.

Correctness oracle. We do not judge correctness by final state alone. Every mediated operation is recorded, and after each run we build the workflow-level serialization graph from the actual recorded conflicts and test it for cycles. Two details make the oracle sound rather than approximate. Write events are published when the piece's transaction applies them, so the recorded order is the order the database actually took rather than the order the workflow buffered them; and the events of an aborted optimistic attempt are retracted so they cannot contribute spurious edges. Application invariants are checked separately.

Plan-state concurrency. Plan installation, the frontier and the set of currently executing pieces are guarded by one lock. A piece that is executing is treated as immutable exactly like one that has committed, so a concurrent admission can never regroup a piece that another thread is in the middle of running. An earlier version of this runtime serialized only the admission decision itself and had a race here; the verifier now exercises the concurrent path directly.

5. Verification

Every soundness claim is checked by brute force against enumerated executions rather than argued informally.

Property checked	Scale	Ground truth	Violations
Static certification, 2 workflows	256 plans	all interleavings	0
Static certification, 3 workflows	4096 plans	all interleavings	0
Repair yields a serial-equivalent plan	3820 repairs	all interleavings	0
Cycles persist under refinement (Lemma 2)	1023 refinements	258 sealed cycles	0
Admission soundness (Theorem 1)	3000 revisions	all interleavings	0
Committed prefix never rewritten	3000 revisions	structural equality	0
Sealing characterization (Theorem 2)	9290 choppings	exhaustive future-only search	0
Range footprints catch phantoms	2000 scenarios	all interleavings	0
Concurrent runtime, safe policies	72 threaded batches	trace oracle plus invariants	0
Unchecked control must fail	18 threaded batches	trace oracle plus invariants	0

Table 1. Bounded exhaustive verification. Ground truth enumerates executions and tests conflict-serializability at workflow granularity. The refinement-lemma row is non-vacuous by construction: it only counts scenarios whose sealing genuinely fails to certify.

Static certification is exhaustive over all assignments of read and write to two shared items across two and three workflows: 256 and 4,096 plans, every one of them either certified and checked against all interleavings, or rejected, repaired and then checked against all interleavings of the repaired plan. Prefix-aware admission is checked on 3,000 randomly generated revision scenarios with randomly placed frontiers. For each, we confirm that an admitted plan is serial-equivalent under every interleaving, that the committed prefix was never rewritten, and, by enumerating all 9,290 future-only choppings across those scenarios, that refusal coincides exactly with the non-existence of a certified future-only coarsening. Range footprints are checked on 2,000 mixed key-and-range scenarios. There were no violations of any kind.

Two constructed witnesses are also machine-checked: the impossibility instance of Corollary 1, where the single available future-only chopping fails to certify and the plan admits a non-serializable execution, and the concurrent-admission counterexample of Theorem 3.

The verifier also drives the concurrent runtime rather than only the graph logic. It runs 72 batches of 16 workflows across three workloads and four safe policies at eight threads, with mid-flight revisions in progress, and requires that every recorded history be serializable and every invariant hold; it separately requires that all 18 unchecked-control batches do fail, so a passing run cannot be an artefact of undemanding workloads, and that the manifest check blocks an undeclared write which corrupts shared state when enforcement is switched off.

These results are bounds, not proofs of unbounded correctness. They cover small batches, a two-item alphabet for the static sweep, and syntactic read/write conflicts. They do not establish that real tool manifests are sound, which is an assumption the runtime enforces but cannot originate.

6. Evaluation

All numbers below come from real threaded execution against the mediated SQLite store on Windows 11 ARM64, Python 3.12.10, 12 logical CPUs, with 32 workflows per batch, 8 worker threads, 4 shared pools and 10 seeds per configuration. Reasoning is a 50 ms calibrated delay rather than model inference, so these are mechanism measurements, not end-to-end agent latencies. Throughput runs use synchronous=NORMAL; crash runs use synchronous=FULL.

6.1 Where chopping pays, and where it does not

Workload	Policy	Makespan s	Speedup	p95 s	Non-serial	Invariant fail	Retries
claim	Whole DB transaction	2.08	1.00	1.96	0/10	0/10	0.0
claim	Whole-workflow 2PL	0.67	3.19	0.39	0/10	0/10	0.0
claim	Whole-workflow OCC	0.70	2.99	0.37	0/10	0/10	50.6
claim	Certified chopping	0.28	7.35	0.12	0/10	0/10	0.0
claim	Unchecked chopping	0.30	7.24	0.14	0/10	0/10	0.0
budget	Whole DB transaction	2.07	1.00	1.96	0/10	0/10	0.0
budget	Whole-workflow 2PL	0.69	3.06	0.45	0/10	0/10	0.0
budget	Whole-workflow OCC	0.72	2.90	0.38	0/10	0/10	50.7
budget	Certified chopping	0.68	3.10	0.45	0/10	0/10	0.0
budget	Unchecked chopping	0.29	7.18	0.12	10/10	10/10	0.0
retail	Whole DB transaction	2.09	1.00	1.97	0/10	0/10	0.0
retail	Whole-workflow 2PL	0.68	3.11	0.50	0/10	0/10	0.0
retail	Whole-workflow OCC	0.72	2.93	0.39	0/10	0/10	51.0
retail	Certified chopping	0.69	3.07	0.42	0/10	0/10	0.0
retail	Unchecked chopping	0.28	7.41	0.11	10/10	10/10	0.0

Table 2. Workloads where certification either preserves the proposed chopping (claim) or refuses it (budget, retail). Speedup is relative to the whole database transaction. Non-serial counts runs whose recorded history has a cyclic serialization graph; Invariant fail counts runs whose final state violates the application invariant.

Workload	Policy	Makespan s	Speedup	p95 s	Non-serial	Invariant fail	Retries
calendar	Whole DB transaction	2.10	1.00	1.98	0/10	0/10	0.0
calendar	Whole-workflow 2PL	0.69	3.13	0.44	0/10	0/10	0.0
calendar	Whole-workflow OCC	0.73	2.92	0.41	0/10	0/10	49.8
calendar	Certified chopping	0.70	3.07	0.44	0/10	0/10	0.0
calendar	Unchecked chopping	0.29	7.34	0.12	10/10	10/10	0.0
replan	Whole DB transaction	2.10	1.00	1.98	0/10	0/10	0.0
replan	Whole-workflow 2PL	0.42	5.11	0.21	0/10	0/10	0.0
replan	Whole-workflow OCC	0.28	7.61	0.10	0/10	0/10	1.2
replan	Certified chopping	0.30	7.14	0.12	0/10	0/10	0.0
replan	Unchecked chopping	0.28	7.58	0.11	0/10	0/10	0.0
escalate	Whole DB transaction	2.69	1.00	2.52	0/10	0/10	0.0
escalate	Whole-workflow 2PL	0.90	3.04	0.58	0/10	0/10	0.0
escalate	Whole-workflow OCC	0.61	4.51	0.36	0/10	0/10	14.3
escalate	Certified chopping	0.44	6.24	0.17	0/10	0/10	0.0
escalate	Unchecked chopping	0.46	5.91	0.18	10/10	10/10	0.0

Table 3. Predicate-read (calendar), replanning (replan) and mid-flight escalation (escalate) workloads. Unchecked chopping double-books rooms in every calendar seed despite each agent's logic being individually correct.

The claim workload takes a shared counter, reasons privately, then writes a private result. Certification keeps all 96 pieces and delivers 7.35x over the whole database transaction and 2.35x over whole-workflow two-phase locking, with p95 latency falling from 1.96 s to 0.12 s. This is the case chopping exists for: the shared dependency is short and the private interval is long.

The budget, retail and calendar workloads are the honest negative result. Each has a read-modify-write or scan-then-insert straddling the reasoning gap, so certification correctly merges the pieces and the policy becomes whole-workflow locking: 3.10x, 3.07x and 3.07x, against 3.06x, 3.11x and 3.13x for two-phase locking, a difference within seed-to-seed noise. Certified chopping bought nothing here, and it is important that it bought nothing rather than buying speed unsafely. Unchecked chopping on those same three workloads is roughly 2.4x faster still and is non-serializable with a violated invariant in all 10 of 10 seeds.

The escalate workload, where two thirds of workflows revise their plan mid-flight, is where prefix-aware recertification earns its place: 6.24x versus 3.04x for two-phase locking and 4.51x for optimistic concurrency control, with no violations, while the unchecked variant fails in all 10 seeds.

6.2 Enforcement against a dishonest manifest

We ran six workflows whose declared footprint claims a private write but whose code writes shared state. With enforcement disabled, every write lands, the shared counter is corrupted and the recorded history is not serializable.

With enforcement enabled, all six accesses are refused before touching the database, the shared value is untouched and the history is serializable. The certificate is only as good as the mediation layer, and the mediation layer is where the guarantee is actually produced.

6.3 Admission outcomes for mid-flight revisions

Shared pools	Revisions	Admitted	Admitted coarsened	Refused	Admission rate	Correctness failures
1	220	10	0	210	0.04	0
2	220	20	0	200	0.09	0
4	220	37	0	183	0.17	0
8	220	73	9	138	0.37	0

Table 4. Outcomes of 220 mid-flight revisions per contention level, 10 seeds each. Correctness failures counts runs that were non-serializable or violated an invariant.

Between 4 and 37 percent of revisions are admissible; the rest are refused because the residual cycle passes through a committed piece. This is the most important limitation we measured. Prefix-aware recertification decides the question correctly and cheaply, and it never admits an unsafe revision, but the answer is frequently no. An agent architecture that escalates into shared state after committing a prefix should expect refusal to be the common case, and should be designed to either declare the contingent footprint up front, defer commitment until the plan is settled, or provide compensation. The whole-workflow baselines avoid the problem by reserving the conservative union footprint in advance, which is precisely the cost chopping is trying to avoid.

6.4 Crash and recovery

Arm	Injected fault	State after crash	After recovery	Invariant
Certified chopping + continuation	process dies after a piece commits	pair 0 broken, total 5990	resumed xfer-00, total 6000	restored
Certified chopping, no continuation	identical fault	pair 0 broken, total 5990	no recovery run	violated
Whole DB transaction	crash before commit	total 6000, no progress	rollback only	holds
Certified chopping	crash inside the piece transaction	total 6000	torn piece rolled back	holds

Table 5. Real process kills at chosen boundaries, three workflows moving 10 units between private account pairs. Each pair should total 2000 and the batch should total 6000.

The transfers here have no cross-workflow conflicts, so certification accepts the two-piece chopping. That is the point: the batch is isolation-safe and still not failure-atomic. Killing the process after the debit piece commits leaves a broken pair and a batch total of 5990. Without continuation the damage is permanent. With continuation, recovery resumes the workflow from its durable marker and restores both conservation and completion. A crash inside a piece transaction rolls back the writes together with the progress record, confirming pieces are atomic. The whole-transaction arm rolls back cleanly but makes no progress, which is the trade being made.

6.5 Sensitivity

Axis	Value	claim speedup	budget speedup
think	0	0.59 +/- 0.20	0.82 +/- 0.16
think	0.01	1.37 +/- 0.43	1.06 +/- 0.11
think	0.05	2.29 +/- 0.14	0.97 +/- 0.05
think	0.2	2.62 +/- 0.23	1.00 +/- 0.02
pools	1	5.99 +/- 0.24	1.00 +/- 0.06
pools	2	3.54 +/- 0.31	1.03 +/- 0.05
pools	8	1.55 +/- 0.17	1.02 +/- 0.08
pools	16	1.30 +/- 0.25	1.04 +/- 0.13
threads	1	0.99 +/- 0.01	1.01 +/- 0.01
threads	2	1.22 +/- 0.03	1.01 +/- 0.06
threads	4	1.54 +/- 0.09	1.03 +/- 0.07
threads	16	3.12 +/- 0.58	0.97 +/- 0.03

Table 6. Speedup of certified chopping over whole-workflow two-phase locking, mean and standard deviation over 5 seeds. claim is choppable; budget is not.

The benefit tracks contention structure and available concurrency, not reasoning time alone. With one shared pool the claim speedup is 5.99x, falling to 1.30x with sixteen pools where there was little contention to relieve. With a single worker thread it is 0.99x, rising to 3.12x at sixteen. With zero reasoning time it is 0.59x: chopping is a net loss when there is no private interval to release the lock for, because the extra piece boundaries cost commits. The budget column stays between 0.82x and 1.06x throughout, as it must, since certification always merges it back to whole-workflow locking.

6.6 Certification cost

Workflows	Pieces	Static certify ms	Readmission ms	Merges
8	24	2.0	0.07	8
16	48	10.0	0.26	16
32	96	28.5	0.60	32
64	192	43.5	2.09	64
128	384	131.0	5.00	128

Table 7. Certification and readmission cost. An earlier all-pairs implementation took 22.1 s at 128 workflows; indexing conflict candidates and repairing every mixed block per pass reduced it to 0.13 s.

Cost is not negligible and can dominate. Static certification of the budget workload costs 35.2 ms against a batch whose execution takes 0.68 s, and it purchases no speedup at all, so the correct engineering decision there is to skip chopping entirely. Readmission is much cheaper than initial certification, 5.0 ms at 128 workflows, because the frontier constrains the search. A deployment should cache certificates per program template and recertify only on structural change.

6.7 Real language-model agents

Arm	Certified boundary	Tool calls	Refused	Final counter	Lost updates
A: read and write as separate pieces	uncertified chop	8	0	1/4	3
B: certifier merged the two pieces	read-modify-write sealed	10	6	4/4	0

Table 8. Four concurrent Gemini 3.5 Flash agents per arm, each instructed to reserve exactly one unit, driving the mediated tool as a subprocess. Arm A exposes read and write as separate certified pieces; in Arm B the certifier merged them, so only the atomic operation is inside the manifest.

In Arm A all four agents read zero, all four wrote one, and three updates were lost. In Arm B the enforcement layer refused six calls, including one agent's attempt to write the value 1 after its read was refused, which is exactly the operation that would have caused a lost update. Every agent then discovered the permitted operation from the refusal message and completed its reservation, giving a final counter of 4 with no lost updates. This is a small enforcement demonstration with N=4 per arm and no latency claim, but it exercises the part of the design that a simulation cannot: agents that improvise, and a boundary that holds anyway.

7. Discussion and limitations

The result we would emphasize to a system designer is not the speedup. It is that the admissibility question for mid-flight revisions has a clean answer, that the answer is often no, and that no runtime cleverness changes it once effects are visible. This argues for declaring contingent footprints before committing anything, and for treating the point of first visible write as a commitment boundary in agent design, not merely an implementation detail.

Threats to validity. Reasoning is a calibrated delay in the throughput experiments; real inference has different variance, caching behaviour and cost structure, and an optimistic policy that discards a long generation pays far more than our model charges it. The workloads are small, hand-built and chosen to expose specific graph structures; they are not a measured distribution of production agent traffic, and we do not quantify how often useful safe cuts occur in the wild. Footprints are supplied by the workload rather than inferred from real tool implementations, so we evaluate enforcement of manifests, not their derivation, and in-process workflow bodies are trusted not to bypass the adapter. The store is a single local SQLite database: there are no network partitions, no distributed participants, no irreversible external effects, and no multi-service commit. Verification is bounded and syntactic. The real-agent study has four agents per arm, one model and one task shape, and arm A's exact lost-update count depends on timing.

Relation to prior work. Chopping, semantic conflicts including return values, and the non-removability of cycles under refinement are classical [1], as is the separation of serializability from recoverability [2]. Transaction chains, Callas and IC3 combine static analysis with runtime ordering or dependency tracking to admit executions a static no-cycle test would reject [3, 4, 5]; our criterion is deliberately more conservative and correspondingly cheaper, and their mechanisms suggest a natural way to weaken our refusal rate. Chopping under parallel snapshot isolation targets a different correctness condition and shows how the permitted observer population changes safety [6]. On the agent side, Atomix provides transactional tool use with declared effects and settlement, where the integration layer chooses transaction granularity [7]; OptiMA schedules multi-agent transactions treated as indivisible [8]; CoAgent orders agents and repairs conflicts semantically, assuming notified agents self-heal correctly [9]; concurrency anomalies in multi-agent systems have been formally studied [10]; SagaLLM applies compensation-based decomposition while explicitly relaxing ACID [11]; and Mnemosyne admits untrusted proposals through a deterministic gate [12]. Automatic read-set reconstruction [14] and adaptive control for agent-generated SQL [13] are complementary. What we did not find in that body of work, and what this paper supplies, is the treatment of plan revision over an already-committed prefix, with an admissibility characterization and an impossibility result. That is a search finding, not a priority claim.

Future work. The most valuable next step is to relax refusal using the observed past. Our criterion is static and therefore quantifies over all executions of the revised plan, including executions that the already-fixed history has ruled out. A hybrid criterion that combines the directed dependency graph of the committed prefix with static conflicts over the future should admit strictly more revisions while remaining sound, and Theorem 2 gives the exact

baseline it must beat. Beyond that: inferring footprints from real tool implementations with sound over-approximation, extending the runtime across multiple services with a defined commit protocol, and evaluating on adapted concurrent versions of published agent benchmarks such as tau-bench retail and WorkBench [16, 17], whose isolated task runs do not by themselves create the contention this problem is about.

8. Conclusion

Agents interleave, and the classical answer to interleaving assumes plans that agents do not have. We identified the resulting problem, characterized it completely, and showed that the characterization has teeth: after effects become visible, a large fraction of plan revisions cannot be made safe by any regrouping of the future. We built a runtime that enforces rather than assumes the certificate's hypotheses, verified the mechanism exhaustively against brute-force ground truth, and measured it against baselines with matched guarantees, reporting both the cases where it wins clearly and the cases where the right answer is not to chop at all.

Artifact and reproduction

The artifact contains chopping.py (certification and admission), store.py (mediated durable store), runtime.py (policies and oracle), workloads.py, verify.py, bench.py, agent_tool.py (the cross-process tool used by the agent study), and the complete result files. From the artifact directory: python verify.py reproduces Table 1 in about 3 s, and python bench.py --seeds 10 --count 32 --threads 8 --think 0.05 --pools 4 reproduces Tables 2 to 7 in about 9 minutes. Both use only the Python standard library, create no external effects, and remove their temporary databases. Node.js and the locked docx dependency are needed only to regenerate this document. Timings vary with machine and load; the recorded environment is in results/bench.json. Assertions must remain enabled.

References

[1] Dennis Shasha, Francois Llirbat, Eric Simon, and Patrick Valduriez. Transaction Chopping: Algorithms and Performance Studies. ACM Transactions on Database Systems 20(3), 325-363, 1995. https://doi.org/10.1145/211414.211427

[2] Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman. Concurrency Control and Recovery in Database Systems. Addison-Wesley, 1987. Chapter 2: Serializability Theory. https://www.microsoft.com/en-us/research/wp-content/uploads/2016/05/chapter2.pdf

[3] Yang Zhang, Russell Power, Siyuan Zhou, Yair Sovran, Marcos K. Aguilera, and Jinyang Li. Transaction Chains: Achieving Serializability with Low Latency in Geo-Distributed Storage Systems. SOSP, 276-291, 2013. https://doi.org/10.1145/2517349.2522729

[4] Chao Xie, Chunzhi Su, Cody Littley, Lorenzo Alvisi, Manos Kapritsos, and Yang Wang. High-Performance ACID via Modular Concurrency Control. SOSP, 279-294, 2015. https://doi.org/10.1145/2815400.2815430

[5] Zhaoguo Wang, Shuai Mu, Yang Cui, Han Yi, Haibo Chen, and Jinyang Li. Scaling Multicore Databases via Constrained Parallel Execution. SIGMOD, 1643-1658, 2016. https://doi.org/10.1145/2882903.2882934

[6] Andrea Cerone, Alexey Gotsman, and Hongseok Yang. Transaction Chopping for Parallel Snapshot Isolation. DISC, LNCS 9363, 388-404, 2015. https://doi.org/10.1007/978-3-662-48653-5_26

[7] Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows. arXiv:2602.14849v2, 29 May 2026. https://arxiv.org/abs/2602.14849v2

[8] Umut Calikyilmaz, Nitin Nayak, Jinghua Groppe, and Sven Groppe. OptiMA: A Transaction-Based Framework with Throughput Optimization for Very Complex Multi-Agent Systems. arXiv:2511.03761v1, 5 November 2025. https://arxiv.org/abs/2511.03761v1

[9] Hongtao Lyu, Dingyan Zhang, Mingyu Wu, Xingda Wei, and Haibo Chen. CoAgent: Concurrency Control for Multi-Agent Systems. arXiv:2606.15376v1, 13 June 2026. https://arxiv.org/abs/2606.15376v1

[10] Sajjad Khan. Verified Detection and Prevention of Concurrency Anomalies in Multi-Agent Large Language Model Systems. arXiv:2606.17182v1, 15 June 2026. https://arxiv.org/abs/2606.17182v1

[11] Edward Y. Chang and Longling Geng. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. PVLDB 18(12), 4874-4886, 2025. https://doi.org/10.14778/3750601.3750611

[12] Edward Y. Chang and Longling Geng. Mnemosyne: Agentic Transaction Processing for Validating and Repairing AI-generated Workflows. arXiv:2607.00269v3, 31 August 2026. https://arxiv.org/abs/2607.00269v3

[13] Weixing Zhou, Zhiyou Wang, Zeshun Peng, Hetian Chen, Yanfeng Zhang, and Ge Yu. ATCC: Adaptive Concurrency Control for Unforeseen Agentic Transactions. arXiv:2603.13906v1, 14 March 2026. https://arxiv.org/abs/2603.13906v1

[14] Sajjad Khan. S-Bus: Automatic Read-Set Reconstruction for Multi-Agent LLM State Coordination. arXiv:2605.17076v2, 22 May 2026. https://arxiv.org/abs/2605.17076v2

[15] DBOS. Database Transactions. Official Python documentation, accessed 10 September 2026. https://docs.dbos.dev/python/tutorials/transaction-tutorial

[16] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045v1, 17 June 2024. https://arxiv.org/abs/2406.12045v1

[17] Olly Styles, Sam Miller, Patricio Cerda-Mardini, Tanaya Guha, Victor Sanchez, and Bertie Vidgen. WorkBench: a Benchmark Dataset for Agents in a Realistic Workplace Setting. arXiv:2405.00823, 2024. https://arxiv.org/abs/2405.00823